



Git for Hardware Engineers

Version control for schematics, PCBs, BOMs and firmware

KiCad diffs • BOM history • Releases • Collaboration

Audience: Engineering Students & Junior Engineers

Why • Setup • KiCad + Git • Releases • Best practice



Why Git for Hardware?

Software solved this 20 years ago

History

Every change tracked forever, with who & why.

Collaboration

Multiple engineers work on same project safely.

Releases

Tag every rev — exact state fabbed.

Backup

Distributed = no single point of failure.

Code review

Pull-request workflow for design review.

Open source

Easy to share + accept community contributions.

KiCad + Git — Best Combination

Text-based EDA → great for Git

- ▶ .kicad_sch and .kicad_pcb are plain text S-expression files
- ▶ Diff-friendly: see exactly what changed between revisions
- ▶ Tools: kicad-diff (visual schematic comparison)
- ▶ Libraries (.kicad_sym, .kicad_mod) also text — version them
- ▶ .kicad_pro keeps project settings; commit this too
- ▶ Avoid committing: backup files (-backups/), cache, fp-info-cache
- ▶ Use Git LFS for 3D models (.STEP) — they're binary blobs
- ▶ Altium / Allegro files are binary — diff requires plugins or DXP-Toolbox

A Good Hardware .gitignore

What NOT to track

Copy this into your KiCad project's .gitignore:

```
# KiCad
*-backups/
*.bak
*.kicad_pr1
*.kicad_pro-bak
fp-info-cache
*.kicad_sch-bak
~$*.lck

# Build outputs (regenerable)
gerber/
plot/
bom/
3d/
*-cache.lib

# OS
.DS_Store
Thumbs.db

# Editors
.vscode/
*.swp
```

Hardware Development Workflow

Branches, commits, releases

- ▶ main / master — stable, fabbed-and-working design
- ▶ rev-B / rev-C — work-in-progress next revision
- ▶ feature/<name> — try out an idea (e.g. feature/add-usb-c)
- ▶ Commit messages: 'Add USB-C connector with PD' (verb + what)
- ▶ Squash WIP commits before merging — keep history clean
- ▶ Tag every fabbed revision: v1.0-rev-A, v1.1-rev-B, etc.
- ▶ Each tag = exact fab fileset, traceable forever
- ▶ Branch per design experiment — easy to throw away

Useful Git Commands

Hardware-relevant cheatsheet

Command	Purpose
<code>git init</code>	Start a new repo
<code>git clone <url></code>	Copy an existing repo
<code>git status</code>	See what's changed
<code>git add <file></code>	Stage a change for commit
<code>git commit -m "msg"</code>	Save a snapshot with message
<code>git push origin main</code>	Send to remote (GitHub)
<code>git pull</code>	Bring in remote changes
<code>git log --oneline</code>	View history concisely
<code>git diff HEAD~1 HEAD</code>	Show last commit's changes
<code>git tag -a v1.0 -m "Rev A fab"</code>	Tag a release
<code>git checkout -b feature/x</code>	Create + switch to branch
<code>git merge feature/x</code>	Bring branch into main
<code>git stash</code>	Save WIP without commit

Visual Diff Tools

See schematic / PCB changes graphically

kicad-diff

Renders 2 commits side-by-side.
Open-source.

kicad-cli

Built-in 8+ command line
for rendering / export.

plot-diff

Renders Gerber overlays
between commits.

Web-based tools

Embedded plot rendering
in GitHub Actions.

Allegro Diff

Cadence built-in diff
for Allegro users.

Altium ECO

Engineering Change Order
workflow.

Release Process

From commit to fab

- ▶ Finalise the design — ERC + DRC clean
- ▶ Commit final state with descriptive message
- ▶ Tag: `git tag -a v1.0-rev-A -m "Rev A — first fab"`
- ▶ Push tag: `git push origin v1.0-rev-A`
- ▶ Run KiCad's Gerber export with `kicad-cli`
- ▶ Attach Gerber ZIP + BOM + .pos to GitHub Release
- ▶ Send Release URL to your fab — they pull files directly
- ▶ Document changes in CHANGELOG.md for the next revision

Recommended Repo Structure

Clear layout for hardware projects

```
my_pcb_project/
├── README.md           ← project overview, build instructions
├── LICENSE             ← OSH license (CERN-OHL, MIT, etc.)
├── CHANGELOG.md        ← what changed in each rev
├── .gitignore          ← ignore backups, builds
├── kicad/              ← KiCad source files
│   ├── project.kicad_sch
│   ├── project.kicad_pcb
│   └── project.kicad_pro
├── lib/               ← custom symbols, footprints, 3D
├── docs/              ← schematic PDF, photos, app notes
│   ├── schematic.pdf
│   └── photos/
├── firmware/          ← MCU firmware (or link to separate repo)
├── manufacturing/     ← Gerber, BOM, .pos (generated)
│   ├── v1.0-rev-A/
│   └── v1.1-rev-B/
└── tests/             ← test scripts, bring-up notes
```

Pull-Request Design Review

Software-style review for hardware

- ▶ Designer creates branch and pushes changes
- ▶ Opens a Pull Request (PR) with description and screenshots
- ▶ Reviewer comments inline on schematic / PCB images
- ▶ Discussion → designer addresses, re-pushes
- ▶ CI auto-generates Gerber preview, ERC/DRC results
- ▶ Reviewer approves → merge into main
- ▶ Cleaner than email-based review; full history
- ▶ GitHub / GitLab / Bitbucket all support this flow

Common Mistakes

What hardware engineers often do wrong with Git

- ▶ Committing backup files (-backups/, .bak)
- ▶ Storing large STEP / image files without LFS
- ▶ 'WIP' commits with no message — useless history
- ▶ Working on main branch directly — no review
- ▶ Forgetting to tag the exact state that went to fab
- ▶ Hand-editing generated outputs (Gerber) — overwritten on next build
- ▶ Not pushing — local-only repos lost when laptop dies
- ▶ Conflicts in binary files — get worse over time

GitHub Tools for Hardware

Beyond plain version control

Issues

Bug tracking, design ideas,
field-fault tracking.

Releases

Tag + attached fileset for
every fabbed revision.

Actions

Auto-generate Gerbers,
render 3D, run ERC.

Wiki

Long-form docs, design
notes, history.

Projects

Kanban board for next
rev features.

Discussions

Community Q&A,
support channel.

Quick Start Setup

From zero to versioned project in 10 minutes

```
# 1. Install Git
# Windows: git-scm.com • macOS: brew install git • Linux: sudo apt install git
```

```
# 2. Configure your identity
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

```
# 3. Initialise in your KiCad folder
cd my_pcb_project
git init
```

```
# 4. Add the .gitignore from the previous slide
# (use your favorite editor)
```

```
# 5. Stage and commit
git add .
git commit -m "Initial commit – rev A schematic + PCB"
```

```
# 6. Create a GitHub repo (web UI)
git remote add origin https://github.com/you/my_pcb_project.git
git branch -M main
git push -u origin main
```

```
# Done. You now have version control + remote backup.
```

Key Takeaways

Key takeaways from this lecture

Git from day 1

No project is too small.

KiCad = text + Git

Diff-friendly EDA.

Tag every fab rev

Exact state traceable.

PR for design review

Software-style discipline.

CI for outputs

Auto-Gerber & ERC checks.

Backups for free

Distributed = no lost laptop tragedy.

Questions & Discussion

Ask anything — concepts, projects, sourcing, career.

References & Further Learning

- ▶ Pro Git Book (free, online) — git-scm.com/book
- ▶ Atlassian Git tutorials (free)
- ▶ GitHub Skills — free interactive courses
- ▶ KiCad official docs on version control
- ▶ kicad-diff GitHub project
- ▶ INTI-CMNB / KiBot — KiCad CI tool
- ▶ GitLab CI / GitHub Actions hardware demo workflows
- ▶ Robert Feranec — Git for hardware video

Thank you • Build something this week.